

Assignment No: 06 - Assignment: Naïve Bayes Classification on Iris Dataset

Problem Statement:

1. Implement Simple Naive Bayes classification algorithm using Python/R on iris.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix
```

Step 2: Load the Dataset

```
df = pd.read_csv('/content/Iris.csv')
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	
0	1	5.1	3.5	1.4	0.2	Iris-setosa	
1	2	4.9	3.0	1.4	0.2	Iris-setosa	
2	3	4.7	3.2	1.3	0.2	Iris-setosa	
3	4	4.6	3.1	1.5	0.2	Iris-setosa	
4	5	5.0	3.6	1.4	0.2	Iris-setosa	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Step 3: Dataset Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id              150 non-null   int64
1   SepalLengthCm   150 non-null   float64
2   SepalWidthCm    150 non-null   float64
3   PetalLengthCm   150 non-null   float64
```

```
4   PetalWidthCm    150 non-null    float64
5   Species         150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
df.shape
```

```
(150, 6)
```

```
df.size
```

```
900
```

```
df.columns
```

```
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
      'Species'],
      dtype='object')
```

Step 4: Separate Features and Target

```
X = df.drop(["Species","Id"], axis=1)
y = df["Species"]
```

Step 5: Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 6: Train Naive Bayes Model

```
model = GaussianNB()
model.fit(X_train, y_train)
```

▼ GaussianNB ⓘ ?

```
GaussianNB()
```

Step 7: Predict the Species

```
y_pred = model.predict(X_test)
print(y_pred)
```

```
['Iris-versicolor' 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor'
'Iris-versicolor' 'Iris-setosa' 'Iris-versicolor' 'Iris-virginica'
'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-setosa'
'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-versicolor']
```

```
'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
'Iris-setosa' 'Iris-virginica' 'Iris-setosa' 'Iris-virginica'
'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
'Iris-setosa' 'Iris-setosa']
```

Step 8: Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Step 9: Calculate TP, FP, TN, FN

```
TP = np.diag(cm)
FP = cm.sum(axis=0) - TP
FN = cm.sum(axis=1) - TP
TN = cm.sum() - (FP + FN + TP)
print("TP:", TP)
print("FP:", FP)
print("FN:", FN)
print("TN:", TN)
```

```
TP: [10  9 11]
FP: [0 0 0]
FN: [0 0 0]
TN: [20 21 19]
```

Step 10: Calculate Accuracy, Error Rate, Precision, Recall

```
accuracy = np.sum(TP) / np.sum(cm)
error_rate = 1 - accuracy
precision = TP / (TP + FP)
recall = TP / (TP + FN)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

```
Accuracy: 1.0
Error Rate: 0.0
Precision: [1. 1. 1.]
Recall: [1. 1. 1.]
```

Conclusion

The Naïve Bayes classification algorithm was implemented on the Iris dataset to classify flower species. The performance of the model was evaluated using a confusion matrix and metrics

such as accuracy, precision, and recall. The results show that the Naïve Bayes model performs effectively for classification problems with numerical features.